

File Permissions in Linux

Portfolio Activity — Linux System Administration & Authorization Management

Prepared by: Krishna Chandra GS | Google Cybersecurity Certificate — Course 4

Project Description

As a security professional working with a research team at a large organization, part of my responsibility is to ensure that users are authorized with appropriate file system permissions to maintain system security. In this project, I examined existing permissions on the file system within the `/home/researcher2/projects` directory, identified mismatches between current permissions and the organization's authorization policies, and used Linux commands to modify permissions accordingly. This included updating permissions on regular files, a hidden file, and a subdirectory to ensure only authorized users could read, write, or execute each resource.

Check File and Directory Details

To check the existing permissions of all files and directories — including hidden files — in the `projects` directory, I used the following command:

```
ls -la /home/researcher2/projects
```

The `ls` command lists directory contents. The `-l` flag displays results in long format showing permissions, owner, group, size, and name. The `-a` flag includes hidden files (those beginning with a dot). Together, `ls -la` gives a complete view of all file permissions.

Output:

```
total 32
drwxr-x--- 3 researcher2 research 4096 Oct 20 14:30 .
drwxr-xr-x 3 researcher2 researcher2 4096 Oct 20 14:30 ..
-rw-rw-rw- 1 researcher2 research 1234 Oct 20 14:30 project_k.txt
-rw-r----- 1 researcher2 research 1234 Oct 20 14:30 project_m.txt
-rw-rw-r-- 1 researcher2 research 1234 Oct 20 14:30 project_r.txt
-rw-rw-r-- 1 researcher2 research 1234 Oct 20 14:30 project_t.txt
-rw--w--- 1 researcher2 research 1234 Oct 20 14:30 .project_x.txt
drwx--x--- 2 researcher2 research 4096 Oct 20 14:30 drafts
```

Describe the Permissions String

Using `project_k.txt` as an example, the 10-character permissions string is:

```
-rw-rw-rw-
```

Character(s)	Position	Meaning
-	1st	File type: "-" means regular file; "d" would indicate a directory
rw-	2nd–4th	User (owner) permissions: read (r) and write (w); no execute (-)

rw-	5th–7th	Group permissions: read (r) and write (w); no execute (-)
rw-	8th–10th	Other permissions: read (r) and write (w); no execute (-)

This string shows that `project_k.txt` is a regular file where the user, group, and others all have read and write access. The organization's policy does not allow others to have write access, so this file requires a permission change.

Change File Permissions

The organization's policy states that **others should not have write access to any files**. From the output above, `project_k.txt` has write permission for others (`-rw-rw-rw-`). I used the following command to remove write access for others:

```
chmod o-w project_k.txt
```

The `chmod` command modifies file permissions. `o-w` means: target **others**, - remove, **w**rite permission. To verify the change:

```
ls -la project_k.txt
```

Output:

```
-rw-rw-r-- 1 researcher2 research 1234 Oct 20 14:30 project_k.txt
```

The permissions string is now `-rw-rw-r--` — others now have only read access, which aligns with the organization's policy.

Change File Permissions on a Hidden File

The hidden file `.project_x.txt` has been archived by the research team. The policy requires: **no write access for anyone**, but the user and group should retain read access. The current permissions are `-rw--w----` — the user has write access and the group has write access with no read. I used the following command to fix this:

```
chmod u-w,g-w,g+r .project_x.txt
```

`u-w` removes write from the user. `g-w` removes write from the group. `g+r` adds read for the group. Multiple permission changes are separated by commas in a single `chmod` command. To verify:

```
ls -la .project_x.txt
```

Output:

```
-r--r----- 1 researcher2 research 1234 Oct 20 14:30 .project_x.txt
```

The permissions are now `-r--r-----` — user and group can read, no one has write access, and others have no access. This matches the archiving policy.

Change Directory Permissions

The `drafts` subdirectory belongs to `researcher2` and should only be accessible by that user. The current permissions are `drwx--x---` — the group has execute access, which needs to be removed. I used the

following command:

```
chmod g-x drafts
```

g-x removes execute permission from the group. Execute permission on a directory controls the ability to enter (cd into) that directory and access its contents. Removing it prevents group members from accessing drafts.

To verify:

```
ls -la
```

Output (drafts line):

```
drwx----- 2 researcher2 research 4096 Oct 20 14:30 drafts
```

The permissions are now drwx----- — only researcher2 has read, write, and execute access. The group and others have no access, ensuring the drafts directory is fully private.

Summary

In this project, I examined and updated file system permissions in the /home/researcher2/projects directory to align with the organization's authorization policies. Using `ls -la`, I identified four permission issues: others had write access to `project_k.txt`, the hidden file `.project_x.txt` had incorrect write permissions for both user and group, and the `drafts` directory was accessible to the group. I used the `chmod` command to remove unauthorized write access from files, correct the hidden file's permissions to read-only for user and group, and restrict the `drafts` directory to the owner only — ensuring the principle of least privilege was applied throughout the research team's file system.